

## CSCI 4200: Operating System Engineering

### Project — File System Implementation

**Due Date: May 12, 2026, 23:55**

In this project, you are required to extend the programs provided in **Project\_Prog.zip** by implementing three functions:

- **sys\_mkdir()** and **sys\_rdwrt()** in **fs/main.c** (near the beginning), and
- **sys\_hd\_routine()** in **kernel/hd.c** (near the end).

#### 1. Function Requirements

- **sys\_mkdir()** implements the system call **mkdir()** for creating a directory file,
- **sys\_rdwrt()** handles file data read/write operations.
- **sys\_hd\_roution()** processes pending read/write requests from the disk request queue.

Comments outlining the implementation steps are provided within the source code.

#### 2. System Architecture

The file system follows a three-layer design:

- **File System Layer** – provides system calls (**sys\_open**, **sys\_mkdir**, **sys\_read**, **sys\_write**, **sys\_close**, **sys\_delete**, **sys\_flush**).
- **Block Cache Layer** – manages an in-memory cache of 4KB data blocks using LRU replacement and write-back.
- **Disk Driver Layer** – handles asynchronous I/O requests, hardware interaction, and sector-level reads/writes.

#### 3. Data Flow

With this system, hard-disk data are organized into 4 KB blocks. File system calls (e.g., **read()** and **write()**) are served through the File System Layer (e.g. **sys\_rdwrt()**) and mediated by the Block Cache Layer, which maintains an in-memory cache of 4 KB blocks using LRU replacement and write-back. On a cache hit, data are returned immediately; on a miss, a disk I/O request is enqueued in the Disk Driver Layer, and the calling process yields the CPU (its state changes to Blocked).

Disk requests are serviced by **sys\_hd\_routine()** in the Disk Driver Layer, repeatedly invoked via the user-level task (**iotask.c**). Via **sys\_hd\_routine()**, the IO task dequeues I/O requests and performs block reads/writes by invoking sector-level HD functions asynchronously. The task itself yields the CPU and is reawakened by the HD interrupt handler once the request completes. Finally, the I/O task resumes the calling process, allowing the requested data to be delivered to the user.

For further design details, see **fs-design.pdf** (attached).

## 4. Message Printing in User and Kernel

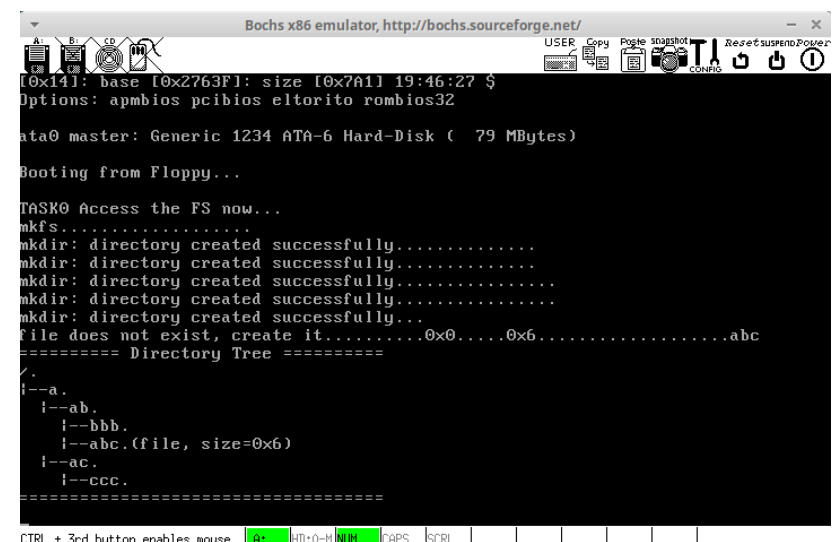
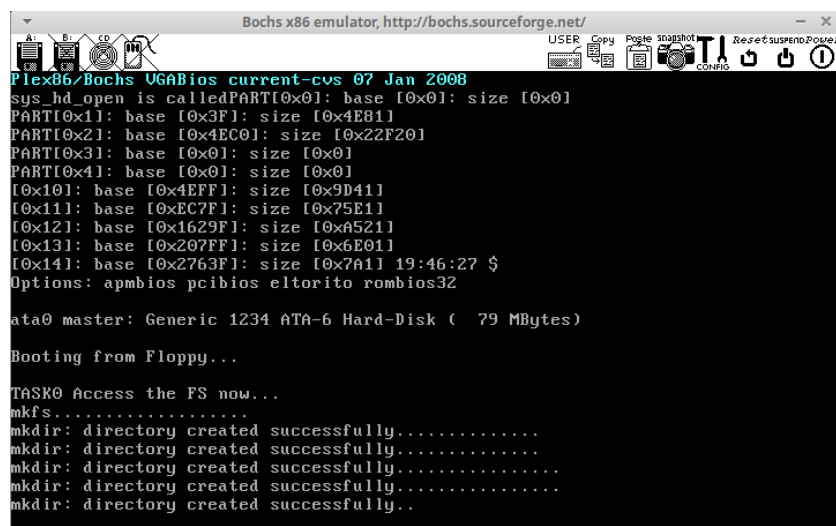
- Kernel Functions
  - `sys_printx()` – prints strings.
  - `sys_write_int_routine()` – prints numbers in hexadecimal format.
- User Functions
  - `printx()` – prints strings.
  - `int_printer()` – prints numbers in hexadecimal format.

**Note:** The printing functions contain known bugs; incorrect or unexpected characters may occasionally appear in the output.

## 5. Grade Policy

Because debugging is challenging and the message-printing functions contain known bugs, the following rule applies:

- 80 marks will be awarded if you successfully integrate the programs with `mytask.c` and `mytask1.c`, producing the specified output as follows:



- **Additional 20 marks** — awarded for integrating new versions of **mytask.c** and **mytask1.c** that contain similar file operations for director/file creations and file data reads/writes.

## 6. Submission Instructions

Submit a single **zip** file that includes the implementation of the three required functions:

- **sys\_mkdir()**
- **sys\_rdwt()**
- **sys\_hd\_routine()**

You may add additional functions if needed to support your implementation.